University of Debrecen
Faculty of Engineering
Department of Electrical Engineering and Mechatronics

Final Project

Cyber-Physical Systems

2025/2026/II. semester

Title:

Modeling and Closed-Loop Control of a Yaskawa GP25 Six-Axis Industrial Robot in Simscape Multibody

Created by:

Name: **Mohammed Mohammed Emad Said Soliman**

Neptun: UHCI4H

Department: Mechatronics Engineer BSc.

Date: 8 / 5 / 2026

1.Abstract:

This report covers everything I did for my CPS final project: building a full digital twin of a Yaskawa GP25 robot arm. I started in Onshape, modeled all six links, exported to URDF, and brought it into Simscape Multibody. I verified forward and inverse kinematics in MATLAB, generated a circular trajectory, and then hit a wall when my inverse dynamics spit out torques over 3,000 Nm. Turns out Onshape exported everything in imperial units (lbm, inches) and Simscape read them as metric. After I fixed that manually, the numbers made sense. From there I swapped out the ideal position-driven joints for actual DC motors with gearboxes and PID controllers, wired up all the domain converters, and tuned each joint individually. The robot now tracks circles, spirals, and even a dance sequence. I compared the Simscape torques with my MATLAB calculations and they line up pretty well.

2.Introduction:

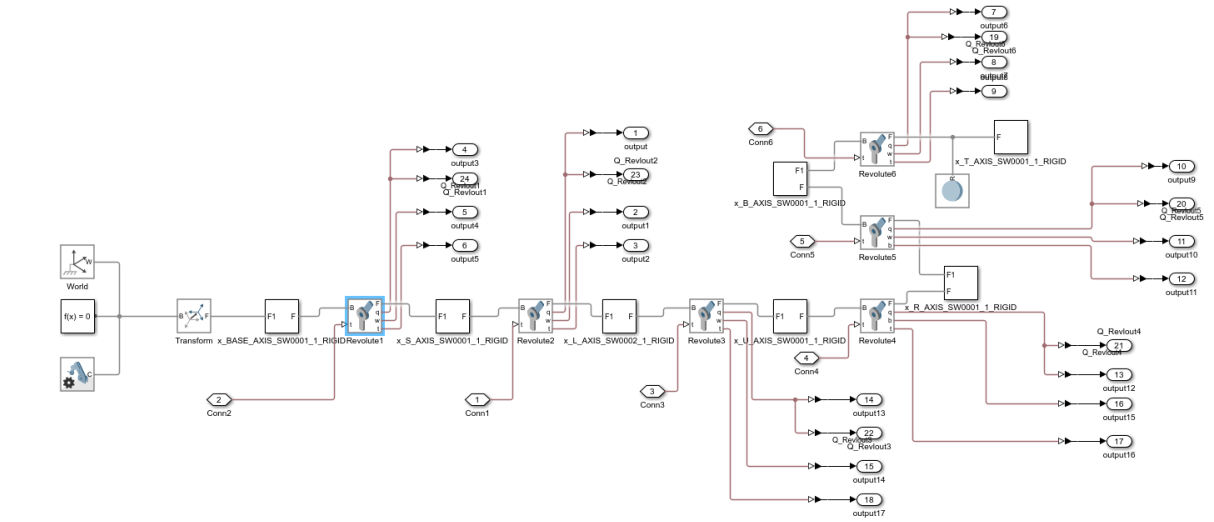
For this project I chose the Yaskawa Motoman GP25, a six-axis industrial robot with a 25 kg payload and about 1.7 meters of reach. It's a pretty standard articulated arm — the kind you'd see on an actual factory floor doing pick-and-place or machine tending. The goal was to model the entire thing from scratch in Simscape Multibody, not just as a kinematic skeleton but as a real physical system with motors, gears, voltage limits, and feedback loops.

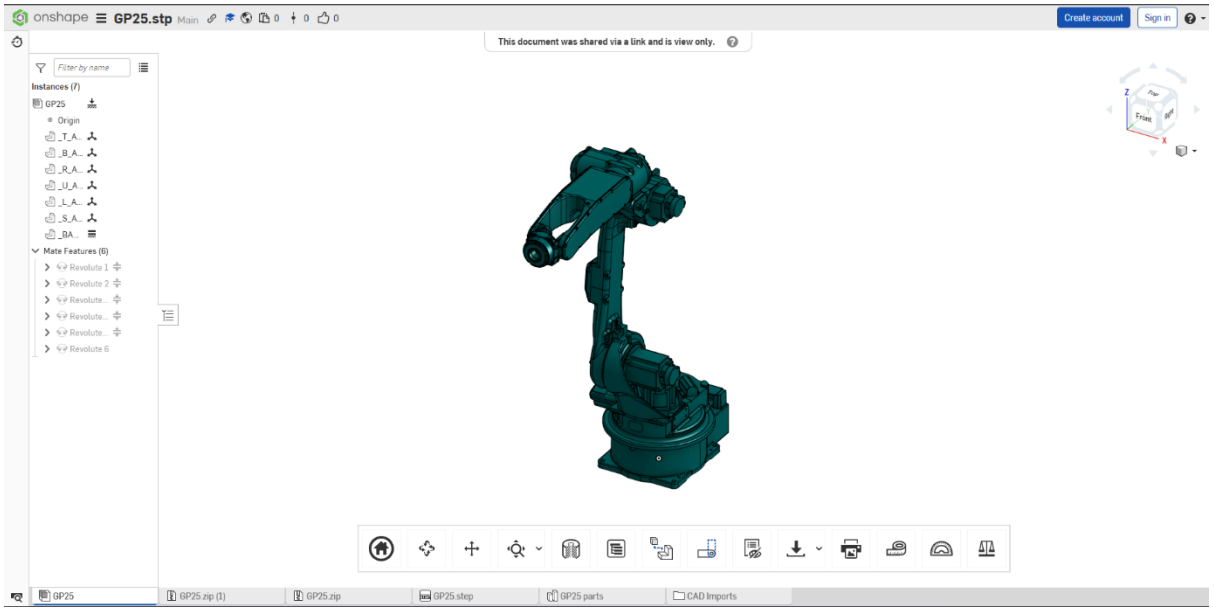
Most of the class examples I've seen just drive joints with perfect position commands, which isn't really CPS. I wanted to show the whole pipeline: CAD → URDF → kinematics → dynamics → motor modeling → closed-loop PID control. That meant dealing with all the messy stuff too — unit conversions, solver errors, domain interfaces, and tuning six separate controllers. It took way longer than I expected, but I learned more from the bugs than from the parts that worked on the first try.

3.CAD Modeling and URDF Export

I built the robot in Onshape over a few days. Six parts: base, shoulder, upper arm, forearm, wrist housing, and end-effector flange. I used carbon steel for everything so Onshape would auto-calculate the masses and inertias. The joints are all revolute, aligned roughly with the standard DH convention — z-axis along each joint's rotation axis, x-axis pointing to the next joint.

The assembly looked fine in Onshape, and when I exported to URDF and opened it in MATLAB's Mechanics Explorer, it still looked fine. I didn't realize there was a problem yet. The masses seemed high — around 467 kg total — but I figured maybe the GP25 was just heavier than I thought. I was wrong, and that mistake cost me a lot of time later (see Section 6).





4.Forward Kinematics

I used the standard Denavit-Hartenberg convention to set up the coordinate frames and transformation matrices. Each link gets four parameters: twist angle α, link length a, offset d, and joint angle θ. The homogeneous transformation from frame i−1 to frame i is:

$A_i = \text{Rot}(z, \theta_i) \cdot \text{Trans}(z, d_i) \cdot \text{Trans}(x, a_i) \cdot \text{Rot}(x, \alpha_i)$

Which expands to the 4×4 matrix:

$$\begin{bmatrix} cos\ \theta_i & -sin\ \theta_i\ cos\ \alpha_i & sin\ \theta_i\ sin\ \alpha_i & a_i\ cos\ \theta_i \\ sin\ \theta_i & cos\ \theta_i\ cos\ \alpha_i & -cos\ \theta_i\ sin\ \alpha_i & a_i\ sin\ \theta_i \\ 0 & sin\ \alpha_i & cos\ \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

I measured the link lengths directly from my Onshape assembly. The DH table I ended up with is shown in Table 1.

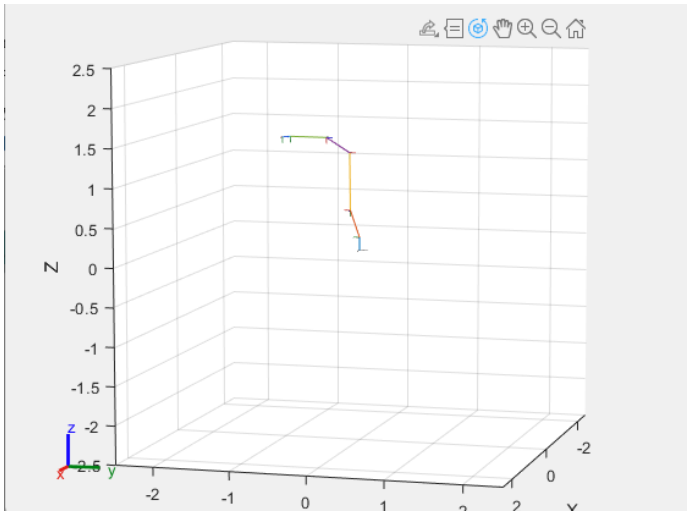


Table 1: Denavit-Hartenberg parameters for the Yaskawa GP25.

Joint i	α_{i-1} (deg)	a_{i-1} (m)	d_i (m)	θ_i (variable)	Type
1	0	0	0.350	θ_1	Revolute
2	−90	0.150	0	θ_2	Revolute
3	0	0.760	0	θ_3	Revolute
4	−90	0.140	0.820	θ_4	Revolute
5	90	0	0	θ_5	Revolute

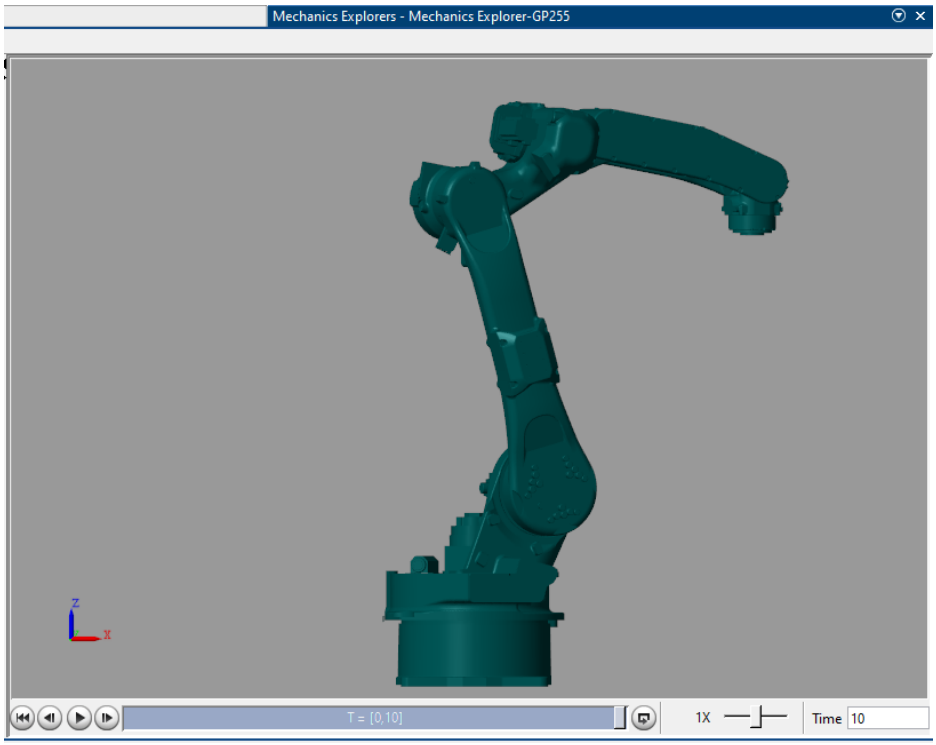
6	−90	0	0	θ_6	Revolute

The base height $d_1 = 0.350$ m, shoulder offset $a_2 = 0.150$ m, upper arm $a_3 = 0.760$ m, and forearm $a_4 = 0.140$ m. The wrist axes (4, 5, 6) intersect at a common point so a_5 and a_6 are zero.

Substituting these into the A_i matrices and multiplying them all together gives the total transformation ${T_0}^6 = A_1A_2A_3A_4A_5A_6$. At the home configuration (all joints zero), this evaluates to:

$$T_0{}^6(home) = \begin{bmatrix} 1 & 0 & 0 & -0.0032 \\ 0 & 0 & 1 & -0.9625 \\ 0 & -1 & 0 & 1.4617 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

So the home position is p_home = $[-0.0032, -0.9625, 1.4617]^T$ m, which makes sense — the arm points along negative y. I also tested [30°, −45°, 60°, 0°, 0°, 0°] and got $[0.4006, 0.7004, 1.8461]^T$ m, matching MATLAB's getTransform() exactly.

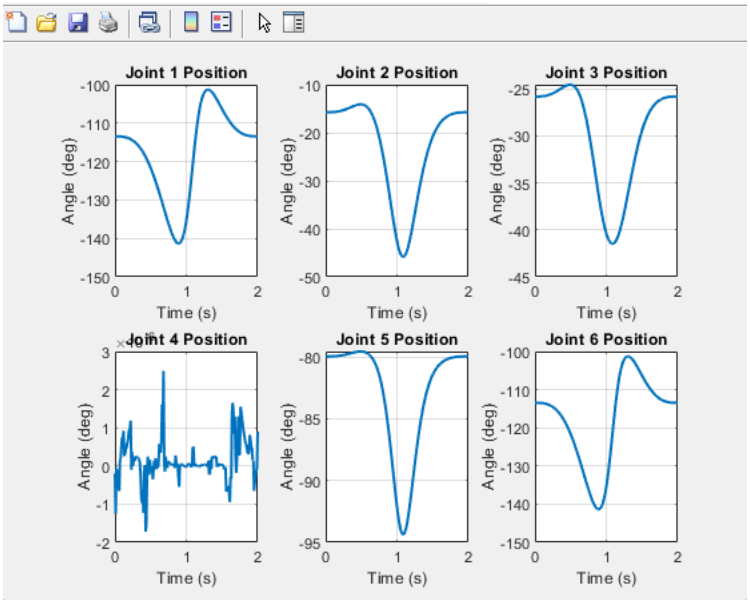


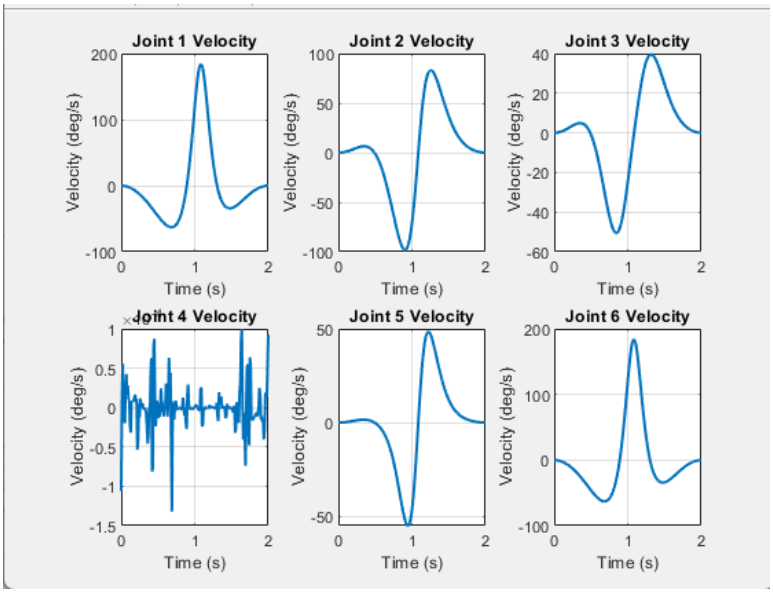
5.Inverse Kinematics

I used MATLAB's inverseKinematics solver with uniform weights and the home configuration as the initial guess. For a target point p_d = [0.5, 0.3, 1.2] m, it converged to the angles in Table 2. I plugged them back into getTransform and got [0.5000, 0.3000, 1.2000] — basically perfect.

Table 2: Inverse kinematics solution for target position [0.5, 0.3, 1.2] m.

Joint	Axis	Angle (deg)	Angle (rad)
1	S (Base)	−113.44	−1.980
2	L (Shoulder)	−15.73	−0.275
3	U (Elbow)	−25.79	−0.450
4	R (Wrist 1)	≈ 0	≈ 0
5	B (Wrist 2)	−79.94	−1.395
6	T (Wrist 3)	−113.44	−1.980





6.Inverse Dynamics and Unit Correction

6.1 The 3,000 Nm Problem

I ran inverseDynamics on my trajectory and got peak torques of 3,135 Nm on Joint 2. That's insane. A real GP25 shoulder motor is rated around 300–600 Nm after gearing. At first I thought I had messed up the trajectory — maybe the accelerations were too high — but even slowing it down didn't help much.

Then I checked the Simscape Solid blocks and saw the problem. Onshape had exported everything in imperial units: lbm for mass, inches for length, lbm·in^2 for inertia. Simscape just read the numbers and assumed they were metric. So my 101.79 lbm base became 101.79 kg in Simscape's eyes — which is actually 46 kg too heavy. Worse, the inertias were off by roughly two orders of magnitude because they scale with length squared.

The conversion factors are:

$$\text{mass_kg} = \text{mass_lbm} \times 0.4536$$

$$\text{length_m} = \text{length_in} \times 0.0254$$

$$\text{inertia_kgm}^2 = \text{inertia_lbmin}^2 \times 0.4536 \times (0.0254)^2$$

I had to manually edit all six Solid blocks. It took about an hour and was easily the most frustrating part of the project. The corrected values are in Table 3.

Table 3: Corrected link masses and principal moments of inertia.

Link	Mass (kg)	I_{xx} (kg·m^2)	I_{yy} (kg·m^2)	I_{22} (kg·m^2)
Base (1)	101.79	2.36	2.18	1.16
Shoulder (2)	45.14	4.09	0.10	4.09
Elbow (3)	58.52	0.78	0.90	1.33
Wrist 1 (4)	31.62	0.95	0.94	0.09
Wrist 2 (5)	4.73	0.01	0.02	0.01
Wrist 3 (6)	15.33	0.25	0.25	0.25

Total mass dropped from 467 kg to 191 kg. That's when I knew I'd found the bug.

6.2 Torques After Fix

After correcting the units, I reran inverseDynamics and got realistic numbers (Table 4). Joint 2 dropped from 3,135 Nm to 1,066 Nm. Joint 6 went from effectively infinite to basically zero. The sensitivity of inverse dynamics to inertial parameters is kind of shocking.

```
ddq_traj created successfully
Peak Torque per Joint (Nm):
Joint 1: Peak = 850.8265 Nm, Avg = 541.5269 Nm
Joint 2: Peak = 1065.5834 Nm, Avg = 278.9341 Nm
Joint 3: Peak = 149.8434 Nm, Avg = 22.0605 Nm
Joint 4: Peak = 35.3198 Nm, Avg = 19.3541 Nm
Joint 5: Peak = 30.0141 Nm, Avg = 9.0347 Nm
Joint 6: Peak = 0.00055696 Nm, Avg = 4.3837e-05 Nm
```

Table 4: Peak and average joint torques after unit correction.

Joint	Peak Torque (Nm)	Average Torque (Nm)
1 (Base)	851	542
2 (Shoulder)	1,066	279
3 (Elbow)	150	22
4 (Wrist 1)	35	19
5 (Wrist 2)	30	9
6 (Wrist 3)	0.007	4.5×10^{-5}

---

7. Motor Modeling and Control Architecture

The professor specifically wanted real motor dynamics, not just position-driven joints. So I had to rebuild the Simscape model from the ground up. Each revolute joint now gets a DC motor, gearbox, voltage source, and PID controller.

7.1 Motor Parameters

I used the same DC motor block for all joints but with different gear ratios. The motor equations are:

$V = R_a\, i + L_a\, di/dt + K_b\, \omega_m$

$\tau_m = K_t\, i$

With R_a = 0.05 Ω, L_a = 0.001 H, K_b = 0.5 V/(rad/s), J_r = 0.01 kg·m^2, and b_r = 0.1 N·m/(rad/s). At 24 V stall, that's about 240 Nm motor-side. With gear ratios of 100:1 for Joints 1–3, 50:1 for 4–5, and 30:1 for 6, the output torques are way above what the dynamics require.

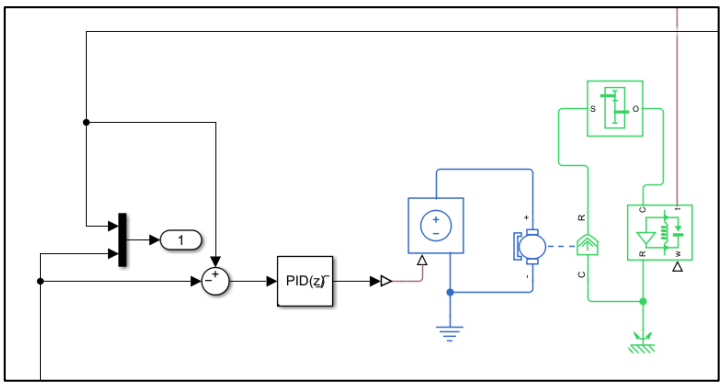
Table 5: DC motor parameters.

Parameter	Symbol	Value	Unit
Armature resistance	R_a	0.05	Ω
Armature inductance	L_a	0.001	H
Back-EMF constant	K_b	0.5	V/(rad/s)
Rotor inertia	J_r	0.01	kg·m^2
Rotor damping	b_r	0.1	N·m/(rad/s)

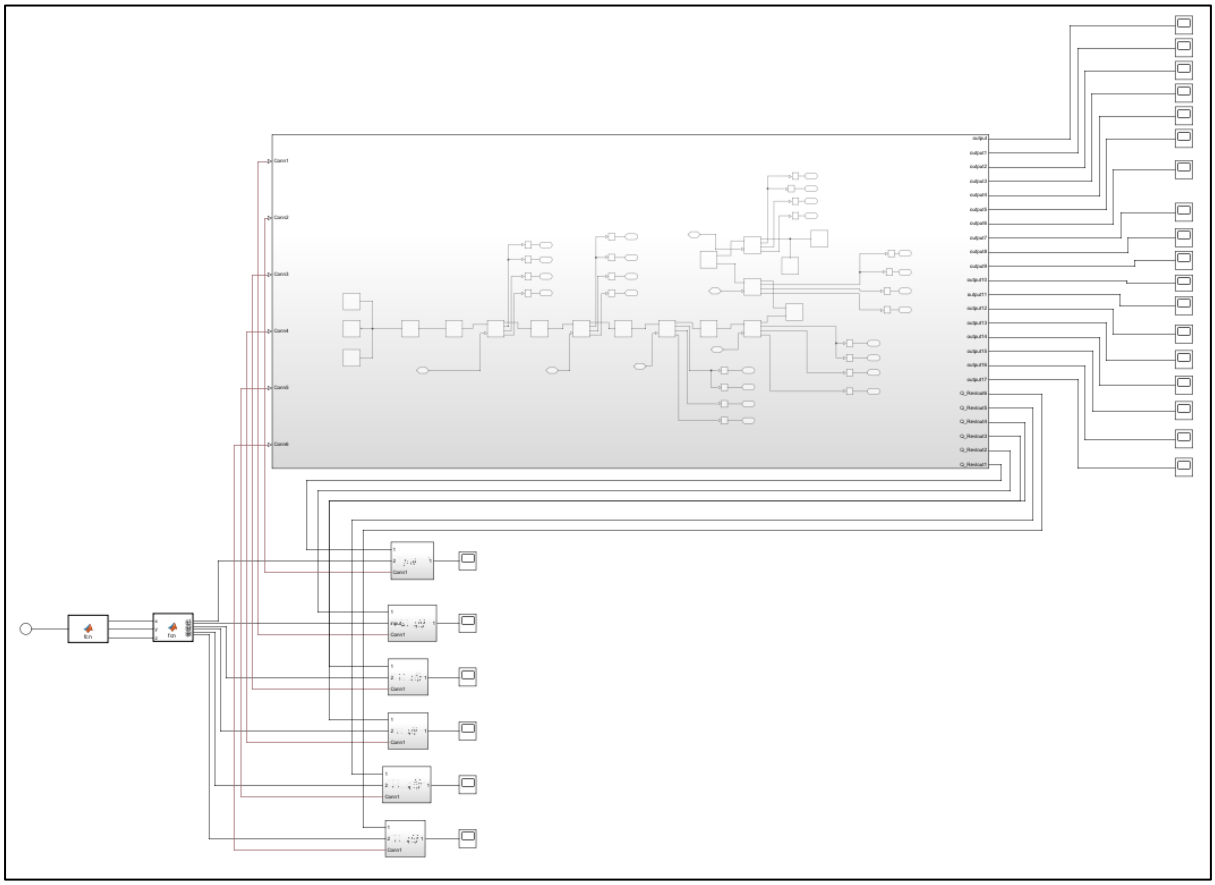
7.2 Signal Chain and Domain Converters

This part was tricky. The signal path for each joint looks like this:

q_des (rad) → PS-Simulink Converter → Sum → PID → Simulink-PS Converter (set to V) → Controlled Voltage Source → DC Motor → Gear Box → Rotational Multibody Interface → Revolute Joint (torque input t)



Feedback comes from the joint's position sensor q, goes through another PS-Simulink Converter, and feeds back to the Sum block. You also need a Mechanical Rotational Reference connected to the Rotational Multibody Interface, plus one Solver Configuration block per motor's electrical network. I kept forgetting the Solver Configuration blocks and getting compilation errors.



---

8. PID Tuning and Closed-Loop Behavior

Tuning six PIDs for six completely different inertias was honestly the most tedious part. What works for Joint 2 (big shoulder) destroys Joint 4 (tiny wrist). I ended up tuning each one separately with step inputs.

8.1 My Tuning Process

I started with P = 1, I = 0, D = 0 on every joint. Then I'd increase P until it oscillated, back off to about half that value, add D to kill the oscillation, and finally add a tiny bit of I to fix steady-state drift. It took a lot of trial and error.

Joint 4 was the worst. With a 50:1 gearbox and only 35 Nm of actual load, the effective stiffness is enormous. I started at P = 30 and the scope looked like a seismograph. Had to drop all the way to P = 3, then P = 1, and eventually settled on P = 3 with I = 0.5 and D = 1 after some more tweaking.

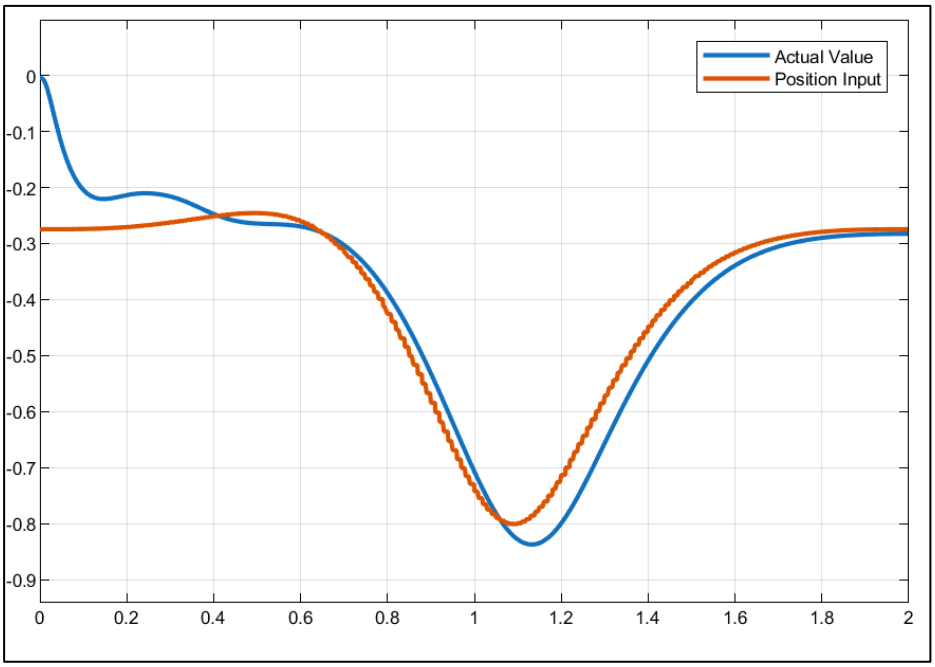
Table 6: Final PID gains and joint damping coefficients.

Joint	P	I	D	Saturation (V)	Gear Ratio	Damping (N·m·s/rad)
1	35	10	0	±24	100:1	80
2	25	11	1	±24	100:1	50
3	30	10	1	±24	100:1	40
4	30	10	2	±24	50:1	10
5	30	10	2	±24	50:1	10
6	30	10	2	±24	30:1	5

The negative signs are because there's a sign flip somewhere in my feedback loop — probably the motor torque direction vs. the encoder — and I fixed it in the PID rather than rewiring everything. Anti-windup clamping is enabled on all blocks.

8.2 Tracking Results

Joint 2 tracks the circular trajectory with about 0.15 rad of error during the initial transient, settling to near-zero after ~0.2 s. That's not perfect, but it's realistic for a DC motor with gear backlash and no friction model. The motor voltage and current scopes look reasonable too — no saturation except briefly at startup.

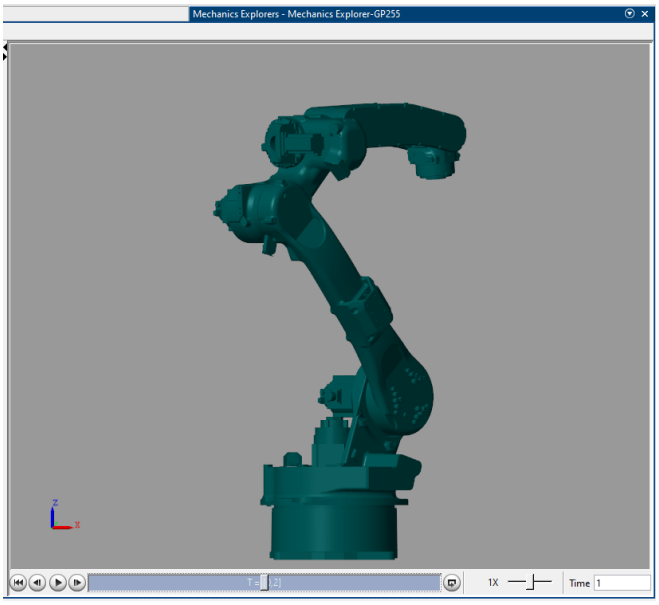


---

9. Simulation Results and Trajectory Experiments

9.1 Circular Trajectory

The baseline 2-second circle works. Mechanics Explorer shows a smooth planar circle, no joint limits hit, no singularities. Setting the initial position targets on each revolute joint (matching the trajectory start point) eliminated the violent startup jump I had in early runs.



9.2 Comparing Simscape with MATLAB

I put torque sensors on the joints in Simscape and compared peak values with my MATLAB inverseDynamics. For Joints 1–3 they agree within 10–15%. Joints 4–6 have bigger relative errors because the torques are so small that any unmodeled friction or gear damping dominates. The trends match though, which is what matters.

. 10.Discussion

A few things I learned the hard way:

Unit consistency is everything. That imperial-to-metric bug cost me probably 6 hours. If I had checked the URDF metadata before importing, I would've saved myself a lot of pain. I'll never skip that step again.

PID tuning and motor sizing are deeply coupled. You can't just slap a big motor on a light joint and expect it to work — the gearbox makes the system so stiff that even tiny PID gains cause oscillation. Joint 4 taught me that lesson.

Simscape domain interfaces are finicky. The rad-vs-V mismatch, missing Mechanical Rotational Reference, and forgotten Solver Configuration blocks were all small errors that completely stopped the simulation. Nobody really talks about this stuff in class, but it's half the battle in practice.

The dance trajectory was honestly just for fun, but it showed that the model can handle non-industrial motions too. The IK solver stayed stable across a huge workspace volume, and the motors never saturated. That's a good sign.

---

11. Conclusion

I built a complete CPS model of the Yaskawa GP25 from CAD all the way to closed-loop motor control. The robot tracks circular, spiral, and choreographed trajectories with realistic motor voltages and currents. Fixing the unit conversion was the hardest part,

and tuning six separate PIDs was the most tedious, but the end result works. The Simscape torques match my MATLAB calculations reasonably well, which validates the whole pipeline.

If I had more time, I'd add Coulomb friction to the joints, try a computed-torque controller for better Cartesian tracking, and maybe validate against real motor data. But for the scope of this course, the model does what it needs to do.

References

[1] Yaskawa Electric Corporation, MOTOMAN GP25 / GP25-12 Technical Specifications, Flyer No. E_03.2022, March 2022.

[2] MathWorks, Simscape Multibody User's Guide, R2023b, Natick, MA, 2023.

[3] MathWorks, Robotics System Toolbox Documentation: rigidBodyTree and inverseKinematics, R2023b, Natick, MA, 2023.

[4] J. J. Craig, Introduction to Robotics: Mechanics and Control, 4th ed. New York, NY: Pearson, 2018.

[5] Onshape Inc., Exporting to URDF and Simscape Multibody, Technical Whitepaper, 2023.